

Capítulo 2: Variables en Python

Gestión de memoria, referencias dinámicas y manipulaciones de datos esenciales para el desarrollo de software industrial y empresarial.

Curso: Curso Profesional de Python

Unidad: Capítulo 2

Nivel: Desde Principiante a Práctico

Enfoque: Desarrollo de Software & Automatización

1. OBJETIVO DEL CAPÍTULO

El propósito fundamental de este capítulo es desmitificar cómo Python gestiona los datos en memoria mediante el concepto de variables. A diferencia de otros lenguajes de programación, Python aborda las variables de una manera única que es crucial entender desde las etapas iniciales para escribir código eficiente, seguro y escalable.

Al finalizar este capítulo serás capaz de:

- Comprender el modelo mental exacto de las variables en Python como referencias a objetos en memoria.
- Declarar, asignar y reasignar variables utilizando buenas prácticas profesionales (PEP 8).
- Identificar y aplicar los tipos de datos primitivos asociados a las variables (enteros, flotantes, cadenas, booleanos).
- Prevenir, diagnosticar y corregir los errores más comunes relacionados con el nombramiento y tipo de variables.
- Implementar variables dentro de contextos reales como procesamiento de datos de sensores, sistemas de inventario y automatización industrial.

2. INTRODUCCIÓN AL TEMA

Imagina que estás organizando una bodega de herramientas industriales. Tienes cientos de piezas físicas en diferentes estantes. Para poder encontrarlas rápidamente, les colocas una etiqueta con una palabra clave. Si mueves la pieza a un nuevo estante, mueves la etiqueta con ella. En Python, una **variable** funciona exactamente como esa etiqueta.

¿Qué es una Variable? Es un nombre simbólico que actúa como una referencia o puntero hacia un valor almacenado en la memoria RAM del ordenador.

¿Por qué existe y qué problema resuelve? Sin variables, tendríamos que referirnos a los datos mediante sus direcciones numéricas directas de memoria RAM (ej. 0x7fee1180b930) o escribir explícitamente el mismo dato estático cientos de veces. Las variables permiten reutilizar información, hacer el código legible y procesar datos dinámicos que cambian con el tiempo (como la lectura de un sensor o la entrada de un usuario).

3. EXPLICACIÓN DETALLADA: ¿CÓMO FUNCIONAN LAS VARIABLES?

En muchos lenguajes tradicionales (como C o C++), una variable es una "caja" con un tamaño fijo donde se guarda un valor. En Python, **las variables NO son cajas**; son **etiquetas que apuntan a objetos**.



1. Creación del Objeto

Python reserva un espacio en memoria RAM, determina el tipo de dato y guarda el valor (por ejemplo, el número flotante 24.5).

2. Vinculación (Binding)

El operador de asignación = conecta el nombre de la variable (temperatura) con el objeto recién creado en la memoria.

Sintaxis General de Asignación

La estructura básica para definir una variable en Python consta de tres elementos esenciales:

```
# Sintaxis: nombre_variable = valor  
presion_caldera = 101.3  
nombre_sensor = "PT100_Zona A"
```

Explicación paso a paso de la Sintaxis:

- **Nombre de la variable:** Es el identificador simbólico. Debe cumplir con las reglas de sintaxis de Python.
- **Operador de asignación (=):** Le dice a Python que evalúe la expresión a la derecha y la asocie al nombre de la izquierda. No significa "igualdad matemática".
- **Valor:** El dato literal u otra variable cuyo objeto de destino será referenciado.

Término Clave: Tipado Dinámico

En Python no necesitas declarar explícitamente el tipo de dato que almacenará una variable (como `int` o `string`). Python deduce automáticamente el tipo de dato en tiempo de ejecución analizando el valor asignado.

4. EJEMPLOS PROGRESIVOS DE APLICACIÓN

A continuación se detallan ejemplos estructurados desde el uso más elemental hasta contextos de programación profesional para sistemas reales.

Nivel 1: Ejemplo Básico (Concepto Fundamental)

```
# Declaración de variables simples contador = 10 print(contador)
```

Análisis línea por línea:

- Línea 1: Comentario descriptivo ignorado por el intérprete.
- Línea 2: Crea un objeto entero con valor 10 y vincula la variable contador a él.
- Línea 3: Imprime en consola el valor apuntado por contador. Muestra: 10.

Nivel 2: Ejemplo Intermedio (Reasignación y Cambio de Tipo)

```
estado_robot = "Inactivo" print("Estado inicial:", estado_robot) # Reasignación de valor  
estado_robot = "En Operacion" print("Nuevo estado:", estado_robot)
```

Explicación del comportamiento: La variable estado_robot apuntaba inicialmente al texto "Inactivo". En la línea 5, se hace apuntar a un nuevo objeto tipo texto "En Operacion". El objeto original queda huérfano en memoria y el Recolector de Basura (Garbage Collector) de Python se encarga de liberarlo automáticamente.

Nivel 3: Ejemplo Práctico (Monitoreo de Sensor Industrial)

```
# Lectura simulada de sensor de presión en PLC identificador_plc = "PLC_LINA_01" presion_psi = 145.8  
valvula_abierta = True # Procesamiento del mensaje de diagnóstico mensaje_alerta = "Sistema: " + identificador_plc + " | Presion: " + str(presion_psi) + " PSI" print(mensaje_alerta)
```

Resultado esperado: Sistema: PLC_LINA_01 | Presion: 145.8 PSI

Punto crítico: Nota el uso de str(presion_psi). Python previene la concatenación directa de números y textos para evitar errores implícitos de tipo.

5. REGLAS DE NOMBRAMIENTO Y CONVENCIONES (PEP 8)

Para escribir código profesional, es fundamental conocer las reglas obligatorias del lenguaje y las recomendaciones de estilo estandarizadas en la industria.

Reglas Obligatorias (Error de Sintaxis)

- No pueden empezar con números (ej. `1variable` es ilegal).
- No pueden contener espacios en blanco.
- No pueden usar caracteres especiales (@, \$, %, -, &).
- No se pueden usar palabras reservadas (`if`, `for`, `class`, `import`, `True`).
- Sensible a mayúsculas/minúsculas (`data` y `Data` son distintas).

Buenas Prácticas (PEP 8)

- Usar estilo **snake_case** (palabras en minúscula separadas por guion bajo). Ej: `velocidad_motor`.
- Nombres descriptivos y significativos. Evitar `x`, `a`, `temp1` salvo en contadores cortos.
- Usar nombres en inglés o español de forma totalmente consistente en todo el proyecto.
- Las constantes se escriben totalmente en mayúsculas: `VALOR_MAXIMO = 100`.

Tabla Comparativa: Estilos de Nombramiento

Estilo	Ejemplo	Uso en Python	Validez / Recomendación
Snake Case	<code>nivel_tanque_agua</code>	Variables y Funciones	Recomendado Estándar
Camel Case	<code>nivelTanqueAgua</code>	Poco común	Permitido pero no idiomático
Pascal Case	<code>NivelTanqueAgua</code>	Clases	Reservado solo para Clases
Screaming Snake	<code>PRESION_MAXIMA_PSI</code>	Constantes	Estándar para Constantes

6. ERRORES COMUNES CON VARIABLES

1. **NameError: name 'x' is not defined**

Causa: Intentar usar una variable antes de haberla declarado o cometer un error tipográfico en su nombre.

```
# ERROR print(temperatura_ambiente) # NameError: name 'temperatura_ambiente' is not defined
```

Solución: Asegurarse de asignar la variable antes de consumir su contenido y verificar la ortografía exacta.

7. EJERCICIOS PRÁCTICOS PROGRESIVOS

Resuelve los siguientes ejercicios de forma individual para fijar los conceptos aprendidos. No consultes las respuestas hasta intentar realizar la implementación por ti mismo.

Ejercicio 1 – Comprensión Básica: Identificación de Identificadores

De la siguiente lista de nombres de variables, indica cuáles son VÁLIDOS y cuáles son INVÁLIDOS según las reglas de Python. Justifica el motivo de los inválidos:

1. `1er_sensor`
2. `sensor_temperatura_1`
3. `valor-total`
4. `class`
5. `_estado_sistema`

Ejercicio 2 – Aplicación: Intercambio de Variables

Contexto: Tienes dos sensores asignados a las variables `sensor_A = 15.5` y `sensor_B = 98.2`. Escribe un programa en Python que intercambie los valores de ambas variables de modo que `sensor_A` contenga `98.2` y `sensor_B` contenga `15.5`.

Restricción: Intenta realizarlo primero utilizando una variable auxiliar y luego utilizando la sintaxis de desempaqueado múltiple nativa de Python.

Ejercicio 3 – Integración: Tarjeta de Presentación de Telemetría

Define variables para almacenar la siguiente información de un Drone de inspección:

- Modelo del drone (Texto)
- Porcentaje de batería restante (Flotante)
- Altura actual en metros (Entero)
- ¿Está conectado al GPS? (Booleano)

Imprime un reporte en consola estructurado que muestre todos los datos con sus respectivos tipos utilizando la función `type()`.

8. MINI PROYECTO DEL CAPÍTULO

Proyecto: Sistema de Control de Acceso y Reporte de Almacén

Contexto: Te han contratado para desarrollar la lógica base de un terminal de control de stock para una planta automatizada de ensamble de motores.

Requisitos Funcionales:

1. Definir variables de identificación para el operador actual: Nombre completo, ID de empleado y Turno.
2. Registrar la información del componente actual en producción: Nombre de la pieza, Código de lote, Cantidad disponible y Precio unitario en USD.
3. Calcular el valor total en inventario multiplicando la cantidad disponible por el precio unitario y guardarlo en una nueva variable.
4. Generar un estado de alerta booleano: si la cantidad disponible es menor a 20 unidades, la variable `requiere_reorden` debe evaluarse como `True`, de lo contrario `False`.
5. Mostrar el reporte final con formato limpio en la consola.

Estructura Guiada del Código (Paso a Paso):

```
# Paso 1: Datos del Operador nombre_operador = "Carlos Mendoza" id_empleado = 4892 # Paso 2: Datos del Componente nombre_pieza = "Válvula de Admisión T4" cantidad_stock = 15 precio_unitario = 45.50 # Paso 3: Cálculos de Inventario valor_total_inventario = cantidad_stock * precio_unitario requiere_reorden = cantidad_stock < 20 # Paso 4: Salida del Reporte print("== REPORTE DE CONTROL DE STOCK ==") print("Operador:", nombre_operador) print("Pieza:", nombre_pieza) print("Valor Total ($):", valor_total_inventario) print("¿Requiere Reorden URGENTE?:", requiere_reorden)
```

9. COMPRUEBA TUS CONOCIMIENTOS (AUTOEVALUACIÓN)

Responde a las siguientes preguntas para verificar tu nivel de dominio técnico alcanzado en este capítulo:

1. ¿Qué ocurre internamente en la memoria RAM cuando le asignas un nuevo valor a una variable existente en Python?
2. ¿Por qué se dice que Python utiliza un sistema de "tipado dinámico"?
3. ¿Cuál es la diferencia fundamental de convención entre las variables `mi_variable` y `MI_VARIABLE` según PEP 8?
4. ¿Qué tipo de error genera ejecutar la línea de código `print("Edad: " + 25)` y por qué ocurre?
5. ¿Es válido asignarle el valor de una variable a otra variable diferente? ¿Qué sucede con las referencias de memoria?

RESPUESTAS Y EXPLICACIONES

1. **Respuesta 1:** Se crea un nuevo objeto en memoria con el nuevo valor y la variable cambia su referencia (puntero) hacia ese nuevo objeto. El objeto anterior es eliminado posteriormente por el recolector de basura.
2. **Respuesta 2:** Porque no se requiere declarar el tipo de dato al crear la variable; Python lo detecta automáticamente según el contenido asignado.
3. **Respuesta 3:** `mi_variable` sigue el estándar de una variable común modificable, mientras que `MI_VARIABLE` indica por convención que debe ser tratada como una Constante.
4. **Respuesta 4:** Genera un `TypeError`. No se puede concatenar implícitamente un objeto tipo texto (`str`) con un objeto entero (`int`). Solución: `str(25)`.
5. **Respuesta 5:** Sí, es totalmente válido. Ambas variables pasarán a apuntar temporalmente exacto al mismo objeto en memoria RAM.

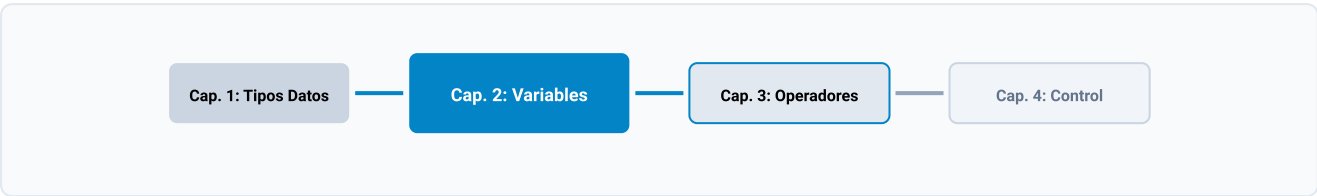
10. RESUMEN DEL CAPÍTULO

Concepto	Para qué sirve	Cuándo utilizarlo	Error Común
Variable	Referenciar valores en memoria RAM.	Siempre que necesites almacenar/reutilizar datos.	Usar variables no declaradas (NameError).
Asignación (=)	Vincular un nombre con un objeto.	Para crear o actualizar el valor de una variable.	Confundir asignación (=) con comparación (==).
Snake Case	Convención de nombres legibles.	En todas las variables y funciones.	Mezclar mayúsculas o usar guiones medios (-).

Ideas Clave que Debes Recordar

- En Python, las variables son **etiquetas que apuntan a objetos**, no contenedores físicos estáticos.
- Python infiere el tipo de dato automáticamente (**Tipado Dinámico**).
- Respeta siempre las directrices de la guía de estilo **PEP 8** para garantizar un código profesional y legible.
- El operador = no evalúa igualdad matemática, evalúa **asignación de referencia** de derecha a izquierda.

11. MAPA DE APRENDIZAJE PROMOCIONAL



Próximo Paso: Ahora que sabes cómo almacenar información en variables, en el **Capítulo 3: Operadores y Expresiones** aprenderás a transformar, comparar y manipular estos datos almacenados para dotar de lógica de cómputo a tus programas.